

Systolic Array CNN/GEMM Accelerator

Project Report — Design Overview and Explanation

Contributors

Name	Role
Adel Wael Adel	Test-bench and Asynchronous FIFO
Yousef Micheal	Software Reference(Python)
Mohammed Atif	Image to column, Control system and Synchronous FIFO
Mohammed Ramadan	The Systolic array and Delay
Kirollos Ibrahim	The Processing element

[Digital Fpga : NTI / Itida]

Contents

1. Introduction to the Systolic Arrays

- Project overview
- Goals and objectives
- Scope

2. Explaining the Functionality of each module

- System overview
- Component-by-component breakdown
- How the parts work together

3. Explaining the Initial Designs and the Reasons They Changed

- First-draft design
- Issues encountered
- Justification for each change

4. Explaining New Problems in the Final Design and How They Were Discovered and Solved

- Problem discovery
- Diagnosis / root cause
- Solution and outcome

1. Introduction to the Project

Project overview:

In this project we have integrated an AI accelerator where it speeds up the CNN(Convolution Neural Networks) process by implementing it in hardware instead of software.

Since the performance of CNN models is limited by the low throughput and computational power of CPU, as well as the low energy efficiency of GPU.

Although Application-Specific Integrated Circuit (ASIC) can achieve low latency and high efficiency, they have poor reconfigurability. When there is a need to update the design, ASIC cannot be reconfigured.

In contrast, Field-Programmable Gate Array (FPGA) are highly programmable and can be customized flexibly according to specific computational requirements.

By writing Hardware Description Language (HDL) code on FPGA, hardware acceleration for specific algorithms and applications can be achieved.

Goals and objectives:

This project focuses on designing a systolic array architecture capable of performing efficient matrix multiplication while leveraging clever data-mapping techniques to also accelerate convolution operations.

Scope:

For this project, the scope is defined around a 3×3 matrix processing elements, with parameterized inputs allowing for flexible configuration, and a FIFO structure at the output to handle streaming results efficiently.

2. Explaining the Function of Everything

System overview

The system is built from a set of interconnected modules, each responsible for a distinct function within the overall architecture:

1. **Processing Elements (PEs)** The fundamental compute units that perform multiply-accumulate operations
2. **Delay Module** Synchronizes data timing across the array
3. **Systolic Array** The interconnected grid of PEs that carries out matrix multiplication
4. **Main Control System** Coordinates data flow and operation sequencing across all modules
5. **Img2Col Controller** Reshapes convolution inputs into matrix form for the systolic array
6. **Output FIFOs** Buffer and stream the final results

Component-by-component breakdown

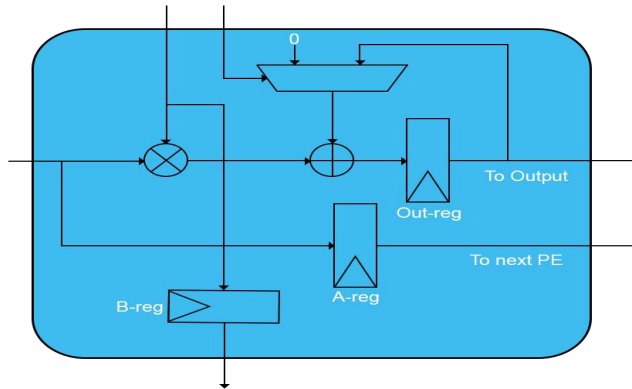
1) **Processing Element (PE)**

A Processing Element, or PE, is the smallest building block of the entire accelerator essentially a single, tiny calculator. Every PE performs the same simple operation, repeated every clock cycle:

- It takes in one number from the left (A) and one number from the top (B).
- It multiplies A and B together.
- It adds this result to a running total stored internally, a process known as accumulation.
- It passes A to the right, to the next PE, and passes B downward, to the PE below it.
- This process is essentially known as MAC(Multiply Accumulate)

This mirrors exactly how matrix multiplication normally by hand. Each output value is the sum of a row multiplied by a column, computed one pair of numbers at a time. Rather than relying on a single calculator to perform these operations sequentially, many small PEs work in parallel, carrying out numerous MAC steps simultaneously this parallelism is what gives the design its speed.

Each PE also responds to two control signals: **valid**, which indicates that the data currently on its inputs is real and should be used, and **hold**, which signals the PE to pause and skip updating its accumulated total for that cycle. Together, these signals give the rest of the design precise control over exactly when each PE is active. This hold signal is mainly utilized by the img2col.



2) Delay Module

The Delay Module is a simple but essential component whose job is to delay incoming signals so that data arrives at different parts of the systolic array in perfect alignment. In a systolic array, data must be staggered as it enters - without this timing offset, values would arrive at each Processing Element out of sync, and the multiply-accumulate operations would produce incorrect results.

For our 3×3 array, this staggering is achieved by applying an increasing number of delay stages to each successive row: the first row of inputs enters with no delay, the second row is delayed by one clock cycle, and the third row is delayed by two clock cycles. This ensures that as data flows diagonally through the array, every PE receives its inputs at exactly the right moment relative to its neighbors.

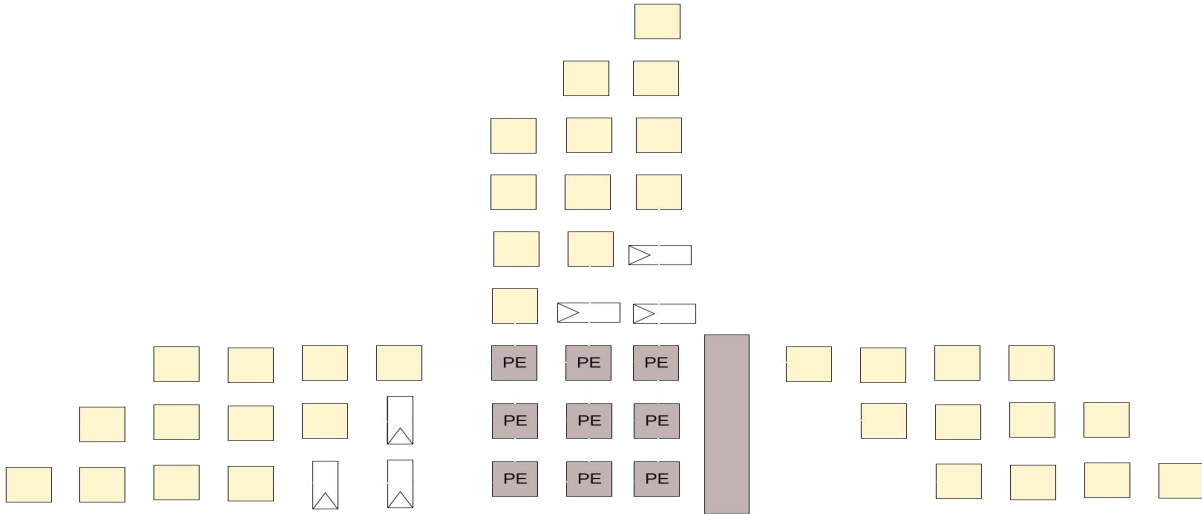
3) The Systolic Array

The systolic array is, at its core, a grid of Processing Elements in this project, a 3×3 grid, giving 9 PEs in total wired together so that data flows through them in a steady, rhythmic pattern. This step-by-step flow, much like blood being pushed through the body in pulses, is where the name (systolic) comes from.

The array multiplies two matrices:- a small image patch (matrix A) and a small weight matrix (matrix B). Rather than feeding these matrices in all at once, their values are streamed in gradually rows of A enter from the left, and columns of B enter from the top, one number per clock cycle. As these values ripple through the grid, each PE multiplies and accumulates them along the way. By the time all the data has flowed through, every PE holds one correct value of the final output matrix.

A key technique makes this ripple effect work correctly: the inputs entering row 1 and row 2 of the array are deliberately delayed by 1 and 2 clock cycles respectively, using small Delay modules. This technique, known as skewing, ensures that each number arrives at exactly the right PE at exactly the right time, properly aligned with the other values it needs to be multiplied against. Without this delay, the results would be incorrect, since numbers from different rows would arrive out of step with one another.

Once the PEs finish accumulating, the completed results are held in the array's (out[row][col]) wires. A set of control signals (ctrl1), (ctrl2), and (ctrl3) then determine which column of results is read out to the next stage, with the output matrix emerging one row at a time.



4) The Control System

The control system is the "brain" of the entire design. It is implemented as a Finite State Machine (FSM), a piece of logic that moves through a fixed set of named states, one at a time, deciding what should happen at each stage. Its purpose is to ensure that every other module in the system, including the PE array and the output FIFOs, operates together in the correct order and at the correct time.

Operation Sequence

In simple terms, the control system performs the following steps, in order:

1. It waits until it receives a signal indicating that new data is ready to be processed.
2. It feeds the input matrices into the systolic array and waits the required number of clock cycles for the PEs to finish their multiply and accumulate operations. This waiting period is tracked using a timer.
3. Once the computation is complete, it selects which column of results to output, one column at a time, using the ctrl1, ctrl2, and ctrl3 signals.
4. For each column being read out, it checks that the corresponding output FIFO is not full, and only then asserts the write enable signal (winc) so the result is safely stored.
5. After all columns have been read out, the system returns to its initial waiting state, ready to process the next matrix, or halts if there is no further data to process.

5) Img2col (Image to column)control

Img2col stands for "image to column." It is a classic and widely used technique for making convolution, the core operation in a CNN, run efficiently on hardware that is optimized for matrix multiplication, such as our systolic array.

The Problem It Solves

A standard convolution operation works by sliding a small filter (in this case, 3x3) across an image, performing a small multiply and add computation at each position. This sliding window pattern does not map

cleanly onto a matrix multiplier, since matrix multiplication expects data in a fixed, structured format rather than a series of overlapping windows.

How Img2Col Solves It

Img2col solves this problem by rearranging the image data before any multiplication takes place. It takes every 3x3 patch that the filter would normally slide over and lays each patch out as a single column of numbers. Once every patch has been converted into a column in this way, the entire convolution operation becomes a single, ordinary matrix multiplication between the rearranged image data and the filter weights, which is exactly the type of operation the systolic array is built to perform efficiently.

Implementation in This Project

In this project, the img2col control module is implemented as its own small FSM, consisting of three states:

1. **Idle** – waiting to begin.
2. **Calc** – actively generating one memory address per clock cycle, systematically walking through every pixel of every sliding window, in order.
3. **Done** – finished processing, waiting to be reset.

The module uses four counters to track its progress. The x and y counters track the current window's position on the image, while the i and j counters track which specific pixel within that window is currently being addressed. During every clock cycle spent in the calc state, the module outputs one valid memory address. This continues until every pixel in every window across the entire image has been covered, at which point the module asserts the done signal.

6) FIFOs Moving Data between 2 stages

A FIFO (First In, First Out) is a small memory queue. Data goes in one end (write) and comes out the other end (read) in the exact same order it went in, like a line of people at a checkout counter. FIFOs are used here to safely hand results from the systolic array to whatever reads them next, without either side having to wait on the other cycle by cycle.

6.1 Asynchronous (Async) FIFO

An asynchronous FIFO is designed for cases where the write side and read side run on two different, unrelated clocks.

Because the clocks are unrelated, an asynchronous FIFO cannot simply compare a write pointer and a read pointer directly. Doing so risks metastability, where a signal is sampled at an unpredictable, invalid value.

To avoid this, an asynchronous FIFO stores its read and write pointers in Gray code, where only one bit changes at a time between counts, greatly reducing the risk of an incorrect value being sampled.

This makes an asynchronous FIFO flexible, since it works regardless of how the two clocks relate to each other. That flexibility comes at a cost, however: added latency before the read side sees new data, plus extra circuitry and complexity from the Gray code logic.

6.2 Synchronous (Sync) FIFO

A synchronous FIFO is simpler: both the write side and read side use the same clock.

With only one clock, there is no risk of metastability between pointers, so no Gray code conversion is needed. Pointers can be compared directly, empty and full flags can be calculated immediately, and data can be read back with far less delay.

We implemented and tested both versions for this project, an asynchronous FIFO and a sync FIFO, and compared the clock cycle differences, the area and the performance between them to confirm which was the better fit for this design.

How the parts work together

Although each module has a distinct job, the real strength of this design comes from how they interact as a single pipeline, moving data from raw image input to a finished convolution result.

The process begins with the **Img2Col Control** module, which walks through the input image window by window, pixel by pixel, generating one memory address per clock cycle. Its job is to, effectively convert the convolution problem into a matrix multiplication problem before any math takes place.

Once the data is in this rearranged form, it is fed into the **Systolic Array**, the 3x3 grid of **Processing Elements** that performs the actual multiply and accumulate work. Rows of the image data and columns of the filter weights stream into the array one value per cycle. To ensure these values arrive at the correct PE at the correct time, the **Delay Module** staggers the second and third rows of input by one and two clock cycles respectively. This skewing is what allows the diagonal, rippling flow of data through the array to produce mathematically correct results.

Coordinating all of this is the **Main Control System**, the FSM that acts as the system's brain. It waits for new data, tracks how many cycles the array needs to finish computing, and then selects which column of results to read out, one at a time, using the ctrl1, ctrl2, and ctrl3 signals. Before each column is read out, the control system checks that the destination FIFO is not full, only asserting the write signal when it is safe to do so.

Finally, the **Output FIFOs** receive these finished results and hold them in order, ready to be read out by whatever stage comes next.

3. Explaining the Initial Designs and the Reasons They Changed

First-draft design

You're going to find the first draft design in this repository.

Issues encountered

Here in the first design we encountered 2 main problems

1. An Oversimplified Control System

The initial control system was far too simple. It merely waited for a fixed number of clock cycles and then output each result in sequence. This approach broke down once `Img2Col` was introduced, since `Img2Col` requires all columns to be properly aligned before being read out, something this simple wait-and-output scheme could not guarantee. The waveform below illustrates this misalignment, showing clearly that this design did not produce the correct output behavior.



2) Limitations of the Asynchronous FIFO

The initial design also used an asynchronous FIFO, primarily to handle Clock Domain Crossing (CDC). While functionally useful for that purpose, it introduced two significant drawbacks that ultimately led to it being discarded in the later design:

- **Large area overhead**, due to the additional Gray code and synchronization logic required.
- **Extra latency**, since the 2-stage flip-flop synchronizer added an additional clock cycle before data could be reliably read.

4. Explaining New Problems in the Final Design and How They Were Discovered and Solved

Problem discovery

The Final Design

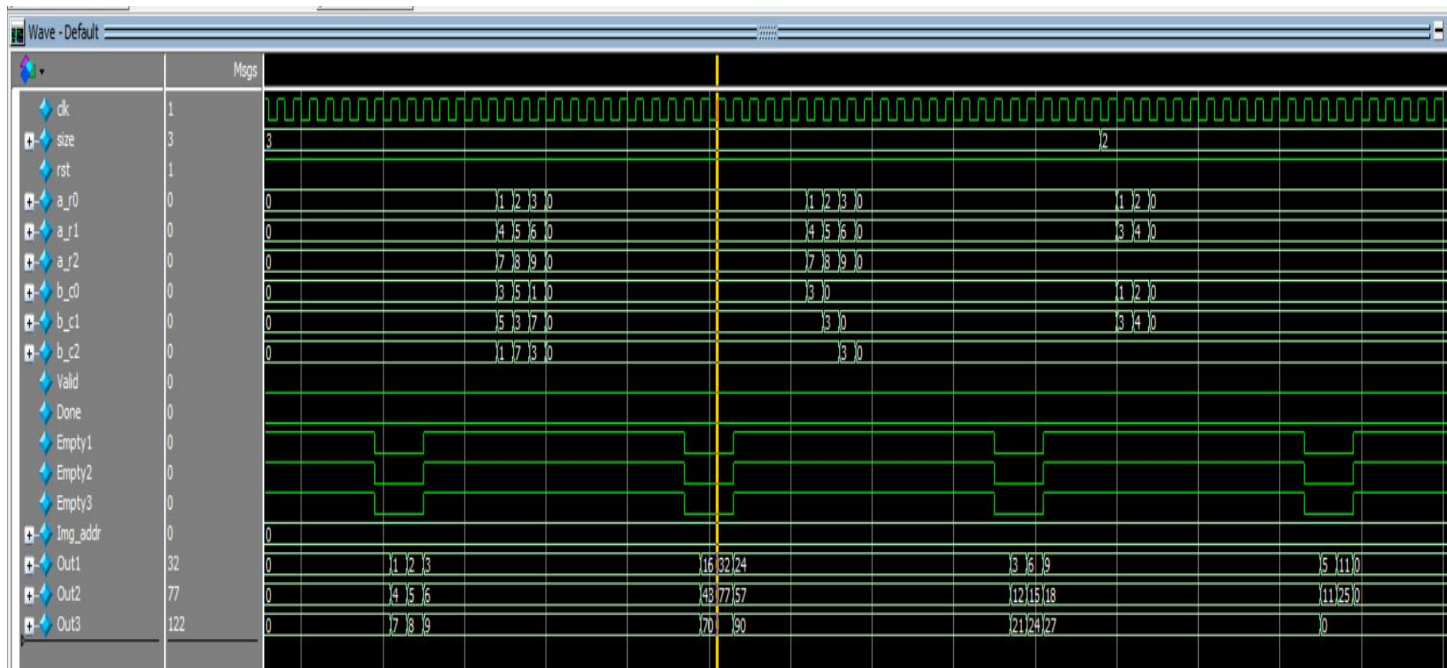
In the final design file, we implemented the remaining core components: the Img2Col controller was completed, the control system (FSM) was finished, and a synchronous FIFO was added at the output stage.

Verification: Matrix Multiplication

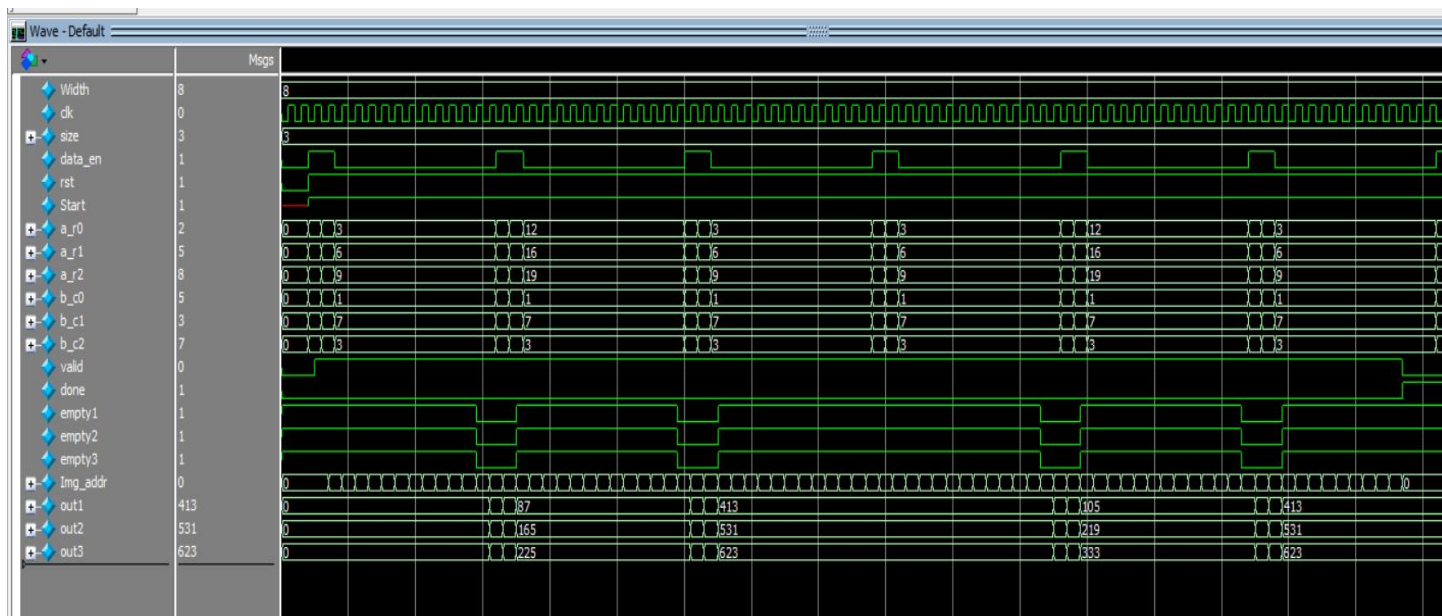
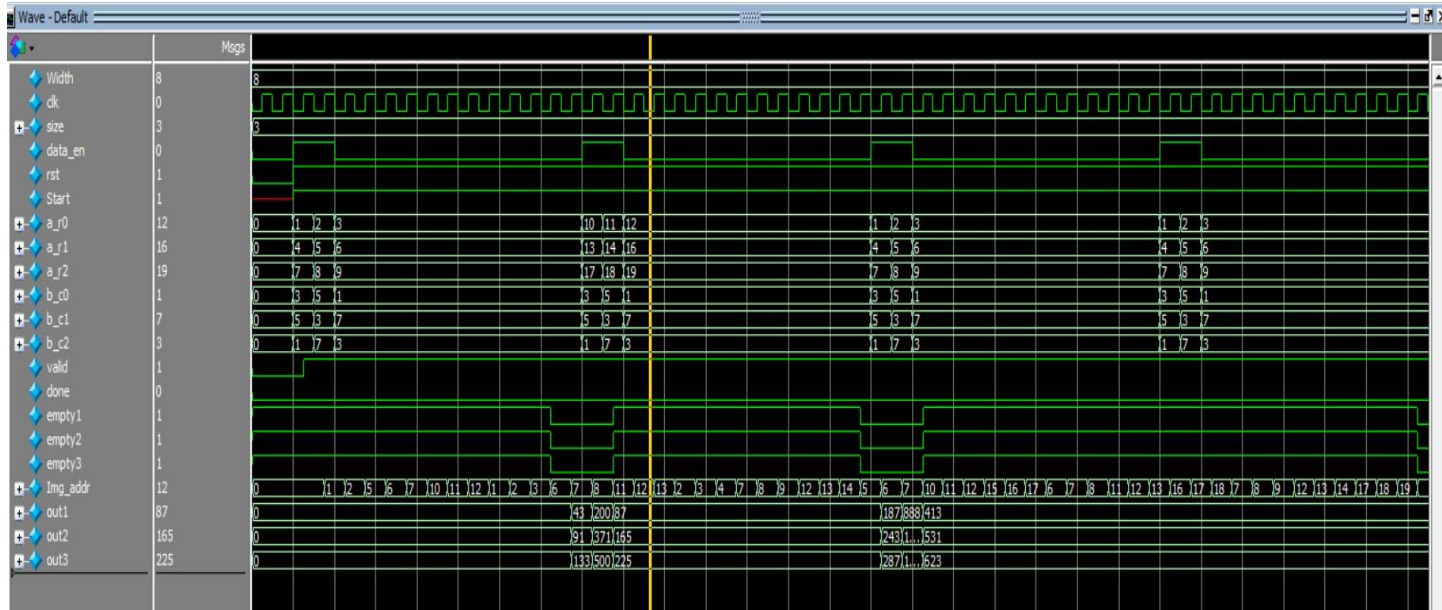
The image below shows the output of a standard matrix multiplication test. The individual input and output values may not be clearly visible in the waveform image itself, but the testbench makes this apparent in simulation. This testbench is available in the GitHub repository under the name **mat_tb**.

Verification: Convolution (Img2Col)

Similarly, the convolution outputs may not be immediately visible in the waveform image alone. The corresponding testbenches, **img1_tb** and **img2_tb**, are available in the repository and show these results in full detail.



This image here is simply the output of normal matrix multiplication the inputs and outputs might not be visible here but the test-bench and will make it more apparent and I will upload this image in my github under the name(mat_tb).



Here the outputs also might not be visible so (img1_tb) and (img2_tb) are there names in my repository.

A Subtle Problem: The Empty Flag Timing Issue

At first glance, this design appeared correct, since the final output values matched the expected results. However, closer inspection revealed a core timing issue: the FIFO's **empty flag** was deasserted (going low) too early, and reasserted (going back high) too early as well.

This is a significant problem because the empty flag is typically treated as an asynchronous signal by the reading side: when high, it indicates that the current output data is invalid and should not be read. If the flag toggles at the wrong moments, the read side cannot reliably distinguish valid data from garbage without

additional compensation logic, which is difficult to implement correctly and defeats the purpose of relying on the flag in the first place.

Although the test results looked correct on the surface, this timing flaw meant the FIFO could not be trusted to signal data validity accurately, and it needed to be addressed before the design could be considered complete.

Diagnosis / root cause

Diagnosing the Empty Flag Issue

As discussed above, the root problem was that the FIFO's empty flag was being deasserted and reasserted too early, creating a timing flaw between the write and read sides.

The underlying cause of this issue traces back to how the synchronous FIFO's empty flag is generated: it simply asserts asynchronously as soon as the read and write pointers reach the same address. For anyone familiar with synchronous FIFO design, this behavior is not itself a bug, it is standard, expected behavior. The real issue was that the surrounding design had not been built to account for this natural pointer alignment behavior.

Correctly resolving this would normally mean reworking the data flow to properly account for the misalignment. However, doing so is time-consuming, since it requires redesigning this stage of the system and then testing for any new bugs introduced in the process.

An alternative approach would be to modify the FIFO itself so that the empty flag asserts one cycle later. While simpler in concept, this too introduces its own risks and potential bugs, making it similarly time-consuming to implement and verify safely and in a timely manner.

This raised an important question: given the time constraints, was there a simpler way to resolve this issue without a full redesign and in short time but does it correctly without any bugs to rediscover?

Solution and outcome

This seemed like a difficult problem to solve, and in most cases it would have required a significant redesign. However, in this case, an existing component could be reused to resolve it: the asynchronous FIFO.

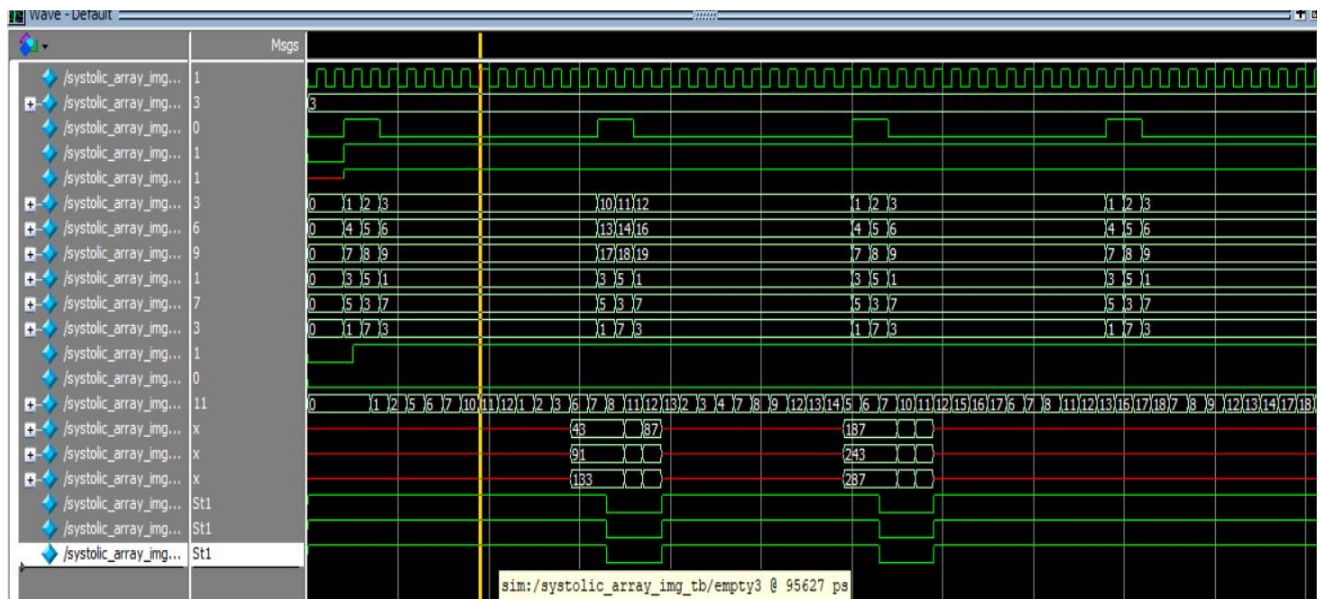
The solution was to reuse the asynchronous FIFO instead of redesigning the timing logic from scratch. This works not because of output ordering, since both FIFO types output data sequentially by definition, but because of how each FIFO generates its empty flag.

In the synchronous FIFO, the empty flag is calculated combinational, directly comparing the read and write pointers within the same clock domain. Since there is no delay in this comparison, the flag toggles the instant the pointers match, which is exactly what caused it to deassert and reassert too early relative to what the downstream logic expected.

In the asynchronous FIFO, the write pointer must first pass through a 2-stage synchronizer before the read side is permitted to compare it against the read pointer. This synchronizer introduces a small, inherent delay before

the read domain registers that a write has occurred, meaning the empty flag on the read side is naturally delayed rather than reacting instantly to pointer equality.

This built-in delay is precisely what resolved the issue. It effectively produces the same delayed empty flag behavior that would have been needed from a manual redesign, except the asynchronous FIFO already provides it as a natural side effect of the synchronization logic required for CDC.



This image will be reuploaded in my repo under the name (async_tb).